



Chapter 1: Introduction:

1. Basic syntax
2. Literal Constants
3. Numbers
4. Variable
5. Basic data types
6. String,
7. Escape Sequences
8. Operators and Expressions
9. Evaluation Order
10. Indentation
11. Input Output
12. Functions
13. Comments.

Introduction to Python Programming

Basic Syntax

Python is a high-level programming language known for its simplicity and readability. It was designed to prioritize code clarity and ease of use, making it an excellent choice for both beginners and experienced developers. Unlike many other programming languages, Python does not use curly braces {} or keywords like BEGIN and END to define code blocks. Instead, Python relies on **indentation** to determine the structure of the code. This unique feature makes Python code visually clean and encourages good coding practices.

Key Characteristics of Python Syntax

Statements: In Python, each statement is typically written on a new line and does not require a semicolon (;) at the end. However, semicolons can be used to separate multiple statements on a single line if necessary.

Whitespace and Indentation: Indentation is mandatory in Python. It defines the scope of code blocks such as loops, functions, and conditionals. Using inconsistent indentation will raise an Indentation Error.

Case Sensitivity: Python is case-sensitive, meaning `Variable` and `variable` are considered two different identifiers.

Comments: Comments begin with a # symbol and are ignored by the interpreter. They are used to make the code more understandable for developers.

Dynamic Typing: Python does not require explicit declaration of variable types. Variables can hold any type of data, and the type can change during execution.

Example: Printing a Message

Below is a simple example to demonstrate Python's syntax:

```
# Python program to print a message

def bgi():
    print("Hello, This is Basal College!")

# Call the function

bgi()
```

Points to be remember for Happy coding:

- The def keyword defines a function named bgi.
 - The code inside the function is indented to show that it belongs to the function block.
 - The print() function outputs the text " Hello, This is Basal College!" to the console.
 - The function is called by writing its name followed by parentheses (greet()).
-

Statements in Python

A statement in Python is an instruction that the interpreter can execute. Examples include assignments, function calls, and loops. Python's syntax is straightforward:

```
# Single-line statement

x = 10

# Multi-line statement using backslash

result = x + \
    20

print(result)    # Output: 30
```

Points to be remember for Happy coding:

- Single-line statements end with a newline character.
 - Multi-line statements can be created using a backslash (\) or by enclosing the expression in parentheses ().
-

Indentation in Python

Indentation is crucial in Python to define blocks of code. All statements in the same block must be indented at the same level. Using inconsistent indentation will result in an error.

Correct indentation

if True:

```
    print("This is indented correctly.")
```

Incorrect indentation

if True:

```
print("This will cause an Indentation Error.")
```

Points to be remember for Happy coding:

- Use 4 spaces per level of indentation (recommended) we can change the size of it by your side.
- Mixing tabs and spaces is discouraged as it can cause errors.

Comments in Python

Comments improve code readability and help other developers understand the purpose of the code.

This is a single-line comment

```
"""          This is a
```

```
multi-line
```

```
comment
```

```
"""
```

```
print("Comments are ignored by the interpreter.")
```

Points to be remember for Happy coding:

- Single-line comments start with #.
- Multi-line comments are enclosed in triple quotes (""" or ''').

Python's Readability and Elegance

Python's syntax emphasizes readability. Here is a comparison to demonstrate how Python differs from a language like C++:

C++ Code:

```
#include <iostream>

using namespace std;

int main() {

    cout << "Hello, World!" << endl;

    return 0;

}
```

Python Code:

```
print("Hello, World!")
```

Points to be remember for Happy coding:

- Python requires fewer lines of code to perform the same task.
- No need for explicit main function or imports for basic functionality.

Advantages of Python Syntax

1. **Ease of Learning:** Python's simple syntax makes it beginner-friendly.
2. **Fewer Lines of Code:** Tasks that take multiple lines in other languages can often be accomplished in one or two lines in Python.
3. **Readability:** Indentation and the lack of unnecessary symbols make Python code easy to read and maintain.

Literal Constants

Literal constants are fixed values directly written into the code that remain unchanged during program execution. Python supports various types of literals, including numbers, strings, Booleans, and special values like None.

Types of Literal Constants

1. **Numeric Literals:** Represent numbers, such as integers, floating-point numbers, or complex numbers.

Examples:

```
x = 42 # Integer literal  
y = 3.14 # Float literal  
z = 2 + 3j # Complex literal
```

2. **String Literals:** Represent sequences of characters enclosed in single ('), double ("), or triple quotes (""" or """).

Examples:

```
name = "Python" # String literal  
multiline = """This is  
a multi-line string."""
```

3. **Boolean Literals:** Represent truth values: True and False.

Examples:

```
is_coding_fun = True # Boolean literal  
is_hard = False
```

4. **Special Literal:** None represents the absence of a value or a null value.

Example:

```
result = None # Special literal
```

Example Program

```
# Demonstrating literal constants  
x = 10 # Integer literal  
y = 3.14 # Float literal  
name = "Ajay Sir" # String literal
```

```
is_happy = True # Boolean literal
```

```
print(x, y, name, is_happy)
```

Output:

```
10 3.14 Ajay Sir True
```

Points to be remember for Happy coding:

- Literals are fixed values that cannot be altered during execution.
- They can represent numeric, textual, Boolean, or special data.
- String literals support various formats for convenience.

Common facts with Literals

1. **Case Sensitivity:** Boolean literals must be written as True and False (not true or false).
2. **Floating-Point Precision:** Be aware of rounding errors in floating-point arithmetic.
3. **Escape Characters in Strings:** Ensure proper handling of special characters like \n or \t in string literals.

Advanced Examples with Literals

Numeric Literals:

Hexadecimal and binary literals

```
hex_num = 0x1A # Hexadecimal
```

```
binary_num = 0b1010 # Binary
```

```
print(hex_num, binary_num) # Output: 26 10
```

String Literals:

Raw strings to ignore escape sequences

```
raw_string = r"C:\Users\Name"
```

```
print(raw_string) # Output: C:\Users\Name
```

Numbers

Python supports three types of numbers: integers, floating-point numbers, and complex numbers. These numbers are immutable, meaning their values cannot be changed once created.

Examples of Numbers

```
# Integer
integer_num = 10

# Floating-point
float_num = 3.14

# Complex number
complex_num = 2 + 3j
```

Points to be remember for Happy coding:

- The `type()` function can be used to determine the type of a number.
- Mathematical operations such as addition, subtraction, multiplication, and division are supported out of the box.
- Python automatically converts between types when performing operations (e.g., adding an integer to a float).

Example with Operations

```
# Arithmetic operations
x = 5
y = 2

print(x + y) # Addition: 7
print(x - y) # Subtraction: 3
print(x * y) # Multiplication: 10
print(x / y) # Division: 2.5
print(x // y) # Floor division: 2
```

```
print(x ** y) # Exponentiation: 25
```

Variables

Variables in Python are used to store data values. They are created when a value is assigned to a name and do not require explicit declaration of their type.

Examples of Variables

```
# Creating variables  
age = 25                                # Integer  
name = "Ajay Wadekar"                 # String  
is_student = True                       # Boolean
```

Points to be remember for Happy coding:

- Variable names must start with a letter or an underscore (_) and can contain letters, numbers, and underscores.
- Variable names are case-sensitive (Name and name are different variables).
- Python is dynamically typed, so the type of a variable can change during execution.

Example Program

```
x = 10  
print(x)                                # Output: 10  
x = "Hello"  
print(x)                                # Output: Hello
```

Basic Data Types

Python supports several built-in data types that can be used to store different kinds of values. The most common types are:

Common Data Types

1. **int**: Represents integer numbers (e.g., 10, -5).
2. **float**: Represents decimal numbers (e.g., 3.14, -0.01).
3. **str**: Represents strings, or sequences of characters (e.g., "Hello").
4. **bool**: Represents Boolean values True and False.

5. **complex**: Represents complex numbers (e.g., $2 + 3j$).

Examples

```
x = 5                # int
y = 3.14             # float
name = "AJAY SIR"    # str
is_sunny = False     # bool
z = 1 + 2j           # complex
```

Points to be remember for Happy coding:

- Use the `type()` function to determine the type of a variable.
- Python automatically assigns the appropriate type based on the value assigned.

Example Program

```
# Checking data types
x = 10
print(type(x))          # Output: <class 'int'>

y = 3.14
print(type(y))          # Output: <class 'float'>

name = "Alice"
print(type(name))        # Output: <class 'str'>

is_sunny = True
print(type(is_sunny))    # Output: <class 'bool'>
```

Strings

Strings are sequences of characters enclosed in single, double, or triple quotes.

Example:

```
message = "Hello, Python!"  
print(message)
```

Escape Sequences

Escape sequences are used to represent special characters within strings.

Example:

```
newline = "This is line one.\nThis is line two."  
tabbed = "Column1\tColumn2"
```

Points to be remember for Happy coding:

- Common escape sequences: `\n` (newline), `\t` (tab), `\\` (backslash).

Operators and Expressions

Python supports arithmetic, relational, logical, bitwise, and assignment operators.

Example:

```
# Arithmetic  
result = 10 + 5  
  
# Relational  
is_equal = (10 == 5)  
  
# Logical  
is_true = True and False
```

Evaluation Order

Python evaluates expressions following the precedence of operators.

Example:

```
result = 10 + 5 * 2 # Multiplication has higher precedence
```

Points to be remember for Happy coding:

- Use parentheses () to enforce evaluation order.

Indentation

Indentation defines the structure and scope of the program.

Example:

```
if True:
    print("This is indented.")
```

Points to be remember for Happy coding:

- Always use consistent spaces or tabs.
- Recommended: 4 spaces per level of indentation.

Input and Output

Python uses `input()` for user input and `print()` for output.

Example:

```
name = input("Enter your name: ")
print("Hello, " + name + "!")
```

Functions

Functions are reusable blocks of code defined using the `def` keyword.

Example:

```
def MyFun (name):
    print(f"Hello, {name}!")

MyFun("Ajay Wadekar Sir")
```

Comments

Comments make the code readable and are ignored by the interpreter.

Example:

```
# This is a single-line comment
```

```
"""This is
```

```
a multi-line comment."""
```

Points to be remember for Happy coding:

- Use # for single-line comments.
- Use triple quotes for multi-line comments.



Bansal Institute of Research Technology & Science, Bhopal
Bansal Institute of Research & Technology, Bhopal
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PRACTICE QUESTION

Write a Python program to calculate the sum of two numbers.

```
num1 = int(input("Enter first number: "))  
num2 = int(input("Enter second number: "))  
print("Sum:", num1 + num2)
```

Write a Python program to find the area of a rectangle given its length and breadth.

```
length = float(input("Enter length: "))  
breadth = float(input("Enter breadth: "))  
print("Area:", length * breadth)
```

Write a Python program to calculate the remainder when one number is divided by another.

```
dividend = int(input("Enter dividend: "))  
divisor = int(input("Enter divisor: "))  
print("Remainder:", dividend % divisor)
```

Write a Python program to find the average of three numbers.

```
num1 = float(input("Enter first number: "))  
num2 = float(input("Enter second number: "))  
num3 = float(input("Enter third number: "))  
print("Average:", (num1 + num2 + num3) / 3)
```

Write a Python program to calculate the result of a number raised to a power.

```
base = int(input("Enter base: "))  
exponent = int(input("Enter exponent: "))  
print("Result:", base ** exponent)
```

Write a Python program to check if a number is divisible by both 3 and 5.

```
number = int(input("Enter a number: "))  
print("Divisible by 3 and 5:", number % 3 == 0 and number % 5 == 0)
```

Write a Python program to check if a number is either positive or divisible by 7.

```
number = int(input("Enter a number: "))  
print("Positive or divisible by 7:", number > 0 or number % 7 == 0)
```

Write a Python program to check if a number is not negative.

```
number = int(input("Enter a number: "))  
print("Not negative:", not number < 0)
```

Write a Python program to check if two numbers are both even.

```
num1 = int(input("Enter first number: "))  
num2 = int(input("Enter second number: "))  
print("Both even:", num1 % 2 == 0 and num2 % 2 == 0)
```

Write a Python program to check if at least one of three numbers is zero.

```
a = int(input("Enter first number: "))  
b = int(input("Enter second number: "))  
c = int(input("Enter third number: "))  
print("At least one is zero:", a == 0 or b == 0 or c == 0)
```

Write a Python program to demonstrate the use of `+=` by adding a number to itself.

```
num = int(input("Enter a number: "))  
num += num  
print("Result:", num)
```

Write a Python program to demonstrate the use of `-=` by subtracting a number from itself.

```
num = int(input("Enter a number: "))  
num -= 5  
print("Result:", num)
```

Write a Python program to demonstrate the use of `\*=` to multiply a number.

```
num = int(input("Enter a number: "))  
num *= 3  
print("Result:", num)
```

Write a Python program to demonstrate the use of `/=` to divide a number.

```
num = float(input("Enter a number: "))  
num /= 2  
print("Result:", num)
```

Write a Python program to demonstrate the use of `%=` to find the remainder.

```
num = int(input("Enter a number: "))  
num %= 3  
print("Result:", num)
```

Write a Python program to perform a bitwise AND operation on two numbers.

```
a = int(input("Enter first number: "))  
b = int(input("Enter second number: "))  
print("Bitwise AND:", a & b)
```

Write a Python program to perform a bitwise OR operation on two numbers.

```
a = int(input("Enter first number: "))  
b = int(input("Enter second number: "))  
print("Bitwise OR:", a | b)
```

Write a Python program to perform a bitwise XOR operation on two numbers.

```
a = int(input("Enter first number: "))  
b = int(input("Enter second number: "))  
print("Bitwise XOR:", a ^ b)
```

Write a Python program to left shift a number by 2 positions.

```
num = int(input("Enter a number: "))  
print("Left shift by 2:", num << 2)
```

Write a Python program to right shift a number by 3 positions.

```
num = int(input("Enter a number: "))  
print("Right shift by 3:", num >> 3)
```

Write a Python program to check if a number is even or odd using a ternary operator.

```
num = int(input("Enter a number: "))  
print("Even" if num % 2 == 0 else "Odd")
```

Write a Python program to check the larger of two numbers using a ternary operator.

```
a = int(input("Enter first number: "))  
b = int(input("Enter second number: "))  
print(a if a > b else b)
```

Write a Python program to check if a number is positive, negative, or zero using a ternary operator.

```
num = int(input("Enter a number: "))  
print("Positive" if num > 0 else "Negative" if num < 0 else "Zero")
```

Write a Python program to assign the smaller of two numbers to a variable.

```
a = int(input("Enter first number: "))  
b = int(input("Enter second number: "))  
smaller = a if a < b else b  
print("Smaller number is:", smaller)
```

Write a Python program to check if a number is divisible by 5 using a ternary operator.

```
num = int(input("Enter a number: "))  
print("Divisible by 5" if num % 5 == 0 else "Not divisible by 5")
```

Write a Python program to check if a number is in a list.

```
lst = [1, 2, 3, 4, 5]  
num = int(input("Enter a number: "))  
print("Number is in the list" if num in lst else "Number is not in the list")
```

Write a Python program to check if two strings are equal.

```
str1 = input("Enter first string: ")
```

```
str2 = input("Enter second string: ")  
print("Strings are equal" if str1 == str2 else "Strings are not equal")
```

Write a Python program to check if a variable is of type integer.

```
var = input("Enter something: ")  
print("Is integer" if isinstance(var, int) else "Not an integer")
```

Write a Python program to concatenate two strings.

```
str1 = input("Enter first string: ")  
str2 = input("Enter second string: ")  
print("Concatenated string:", str1 + str2)
```

Write a Python program to check if an object is not of type float.

```
var = input("Enter something: ")  
print("Not a float" if not isinstance(var, float) else "Is a float")  
  
t by 3:", num >> 3)
```

NOTE PRACTICE PROBLEMS

Basic Syntax

1. Write a Python program to print "Hello, World!".
 2. Identify and correct the syntax errors in the following code:
 3. Write a program that calculates the sum of two numbers using correct syntax.
-

Literal Constants

4. Write a program to store and print the following literal constants: 42, 3.14, "Python".
 5. Determine the data type of the following constants: `True`, `123.456`, `"Hello"`, `None`.
 6. Create a program that uses literal constants to calculate the area of a rectangle (length × width).
-

Numbers

7. Write a program to add two integers and print the result.
 8. Perform basic arithmetic operations (addition, subtraction, multiplication, division) using integers and floats.
 9. Write a program to calculate the square of a given number.
-

Variable

10. Declare a variable to store your age and print its value.
 11. Swap the values of two variables without using a third variable.
 12. Write a program to demonstrate how to update the value of a variable (e.g., incrementing a counter).
-

Basic Data Types

13. Write a program to demonstrate the usage of integers, floats, strings, and booleans.
 14. Convert a given integer to a float and vice versa.
 15. Write a program to check the type of variable using the `type()` function.
-

String

16. Write a program to concatenate two strings and print the result.
 17. Extract and print the first and last characters of a given string.
 18. Reverse a string using slicing.
-

Escape Sequences

19. Write a program to print the following using escape sequences:
 20. Demonstrate the use of the `\\` escape sequence in a string.
 21. Write a program to print the string "Python's syntax is user-friendly!" correctly.
-

Operators and Expressions

22. Write a program to evaluate $5 + 3 * 2$ and explain the result.
 23. Demonstrate the use of logical operators (`and`, `or`, `not`) in Python.
 24. Write a program to find the remainder when a number is divided by another.
-

Evaluation Order

25. Write a program to demonstrate the difference between $(5 + 3) * 2$ and $5 + (3 * 2)$.
 26. Explain the order of evaluation in the expression $5 + 3 * 2 - 8 / 4$.
 27. Write a program to compare two numbers using `==`, `!=`, `<`, and `>` operators.
-

Indentation

28. Correct the following code to ensure proper indentation:
 29. Write a program that uses proper indentation for a nested `if` statement.
 30. Demonstrate the effect of incorrect indentation by writing a sample code snippet.
-

Input/Output

31. Write a program to take a user's name as input and print a greeting message.
 32. Write a program to take two numbers as input and print their sum.
 33. Demonstrate the difference between `input()` and `print()` functions.
-



Functions

- 34. Write a function to calculate the factorial of a number.
 - 35. Write a program to demonstrate the use of a function with parameters and a return value.
 - 36. Create a function that takes a string as input and returns its length.
-

Comments

- 37. Write a program with both single-line and multi-line comments.
 - 38. Use comments to explain the working of a program that calculates the area of a circle.
 - 39. Write a program that is fully commented to demonstrate the importance of documentation.
-

Miscellaneous/Combined

- 40. Create a complete program that takes user input for a name, age, and favorite color, and prints a personalized greeting using functions, comments, and correct syntax.